

CURSO 4: TALLER PRÁCTICO & PROYECTO FINAL INTEGRADOR

CLASE 06 • NIVEL 4 - INTEGRADOR

Clase 06: Testing Riguroso con Pytest, Mocks y Calidad

«Los Tests como el Control de Calidad y Pruebas de Choque de un Vehículo»

Guía de estudio oficial y manual técnico. Diseñado para dominar la programación en Python mediante modelos mentales rigurosos, diagramas de flujo y código de producción.

Autor & Mentor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Python: 3.10+ | **Licencia:** MIT | **wisrovi SUITE**

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre wisrovi SUITE en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Presentación de la Sesión

Bienvenido/a a la **CLASE 06** del **Curso 4: Taller Práctico & Proyecto Final Integrador**. Esta guía técnica está estructurada para proporcionarte las bases conceptuales, arquitectónicas y prácticas indispensables para tu progreso.

🎯 Objetivos de la Sesión

Competencia Conceptual: Comprender la pirámide de testing (Unitarios, Integración, End-to-End), fixtures y mocks con unittest.mock.

Competencia Práctica: Escribir pruebas unitarias que validen endpoints, bases de datos y simulen llamadas a APIs externas sin gastar dinero.

Estructura del Documento

Sección	Contenido Temático	Objetivo Pedagógico
Pág 4: Fundamento	Teoría, Modelo Mental y Metáfora	Comprensión del concepto
Pág 5: Flujo	Diagrama de Arquitectura de Memoria	Visualización del flujo interno
Pág 6: Código	Implementación en Python 3.10+	Patrones idiomáticos (PEP 8)
Pág 7: Gotchas	Antipatrones y Trampas Comunes	Prevención de bugs en producción
Pág 8: Conclusiones	Cierre de Sesión & Notas de Repaso	Consolidación del aprendizaje
Pág 9: Bibliografía	Fuentes Canónicas y Desafío	Ampliación y autoestudio

Clase 06: Testing Riguroso con Pytest, Mocks y Calidad

El testing automatizado es la única garantía de que los cambios nuevos no rompan funcionalidades existentes en producción.

☀ Metáfora Central: Los Tests como el Control de Calidad y Pruebas de Choque de un Vehículo

Hacer tests es como las pruebas de choque de los coches: verificas que los frenos funcionan antes de salir a la autopista.

Principios Teóricos y Modelo Mental

Mocks y Stubs: Simulan respuestas de servicios externos (como APIs de pago o LLMs) para tests rápidos y gratuitos.

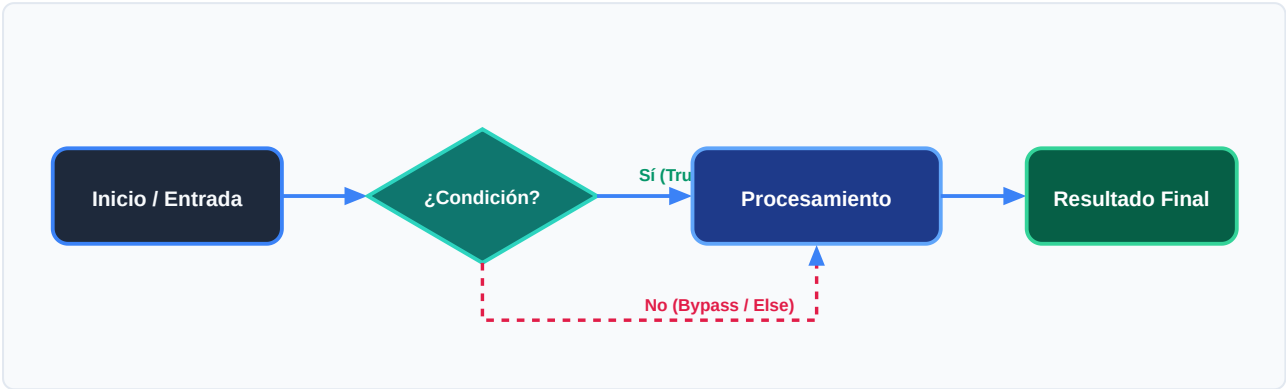
Fixtures de Pytest: Preparan el entorno (bases de datos temporales, clientes HTTP) antes de cada prueba.

⚡ Regla de Oro en Python

Tus tests nunca deben depender de servicios externos reales ni requerir conexión a internet.

Arquitectura y Movimiento de Datos en Memoria

Ejecución de suite de tests, fixtures y reporte de cobertura (Coverage).



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Descubrimiento de archivos test_*.py.	Tests recolectados por pytest.
2. Evaluación	Inicialización de fixtures y mocks.	Entorno aislado preparado.
3. Transformación	Ejecución de assertions (assert a == b).	Verificación de invariantes.
4. Retorno / Salida	Tear-down y emisión de reporte verde (PASSED).	Suite completada.

Visualización Mental

Si un test es difícil de escribir, significa que tu código está demasiado acoplado.

Código de Demostración en Python 3.10+

Tests unitarios con fixtures y TestClient de FastAPI:

main.py (Python 3.10+)

PEP 8 Compliant

```
def calcular_subtotal(items: list[dict]) -> float:
    return sum(i["precio"] * i["cantidad"] for i in items)

def test_calculo_subtotal():
    carrito = [
        {"precio": 10.0, "cantidad": 2},
        {"precio": 5.0, "cantidad": 1}
    ]
    assert calcular_subtotal(carrito) == 25.0

def test_carrito_vacio():
    assert calcular_subtotal([]) == 0.0

print("Ejecutando tests...")
test_calculo_subtotal()
test_carrito_vacio()
print("✅ Todos los tests pasaron exitosamente.")
```

Análisis de Ingeniería del Código

Uso de aserciones simples y cobertura de casos normales y casos límite (carrito vacío).

💡 Buena Práctica de Tipado

El uso sistemático de Type Hints (PEP 484) permite a los editores modernos como VS Code activar autocompletado inteligente y detectar errores antes de la ejecución.

Trampas Frecuentes de Depuración (Gotchas)

Tests frágiles que dependen del entorno.

⚠ Gotcha Frecuente (Trampa de Principiante)

Hacer que los tests llamen a APIs reales falla si no hay internet y consume cuota de pago.

Comparativa: Antipatrón vs Patrón Recomendado

❌ Antipatrón

```
def test_llm():  
    res = llamar_api_real_openai() # ❌ Lento,  
    frágil y cuesta dinero
```

✅ Patrón Correcto

```
def test_llm(mock):  
    mock.patch('llm.call',  
    return_value='Respuesta Mock') # ✅ Rápido y  
    determinista
```

🛡 Consejo de Resiliencia en Producción

Configura coverage para medir qué porcentaje de tu código está cubierto por pruebas.

Conclusiones Clave de la Sesión

Hemos cubierto los pilares teóricos y prácticos de **Clase 06: Testing Riguroso con Pytest, Mocks y Calidad**. El dominio de esta unidad te proporciona una base sólida para afrontar la siguiente semana formativa.

Hitos Alcanzados

Comprensión profunda del modelo mental, capacidad de depuración de gotchas comunes y solidez en la sintaxis idiomática de Python 3.10+.

Notas de Repaso Rápido

Concepto	Punto Clave a Recordar
1. Modelo Mental	Los Tests como el Control de Calidad y Pruebas de Choque de un Vehículo
2. Regla de Oro	Tus tests nunca deben depender de servicios externos reales ni requerir conexión a internet.
3. Gotcha Crítico	Hacer que los tests llamen a APIs reales falla si no hay internet y consume cuota de pago.
4. Buenas Prácticas	Configura coverage para medir qué porcentaje de tu código está cubierto por pruebas.

Mensaje del Instructor

¡Felicitaciones por completar esta lección! Recuerda que la constancia y el pedaleo diario en tu editor de código son los que te convertirán en un programador excepcional.

Fuentes Bibliográficas Oficiales

Para profundizar en los estándares formales del lenguaje y sus implementaciones internas, consulta la documentación canónica:

Recurso Oficial	Descripción	Enlace
Documentación Python 3	Especificación canónica y biblioteca estándar	docs.python.org/3/
PEP 8 — Style Guide	Estándar oficial de formateo y estilo	peps.python.org/pep-0008/
Real Python Tutorials	Patrones de desarrollo e ingeniería	realpython.com
Suite wisrovi en GitHub	Paquetes open source de alto rendimiento	github.com/wisrovi

🏆 Desafío Práctico para el Estudiante

🎯 Reto de la Semana

Escribe un test parametrizado con `@pytest.mark.parametrize` para validar 5 casos de cálculo de IVA.

🔧 Validación Automatizada

Ejecuta `pytest ejercicios/` en tu terminal de VS Code para verificar automáticamente la validez de tu solución.